SUPPLY CHAIN COMMAND CENTER

End-to-End Data Flow

How a raw CSV in `data/input/` becomes a promoted production forecast, an inventory plan, and a chart in the React UI — every stage with the module that runs it, what it reads, and what it writes. Verified against the code on 2026-07-08.

Legend `file / CSV` `python module` `postgres table` `materialized view` `API / UI` `yaml config`

The pipeline spine

Stages 1–5 are data engineering; 6–9 are the ML lifecycle; 10–11 are planning and serving. Two cross-cutting services (MV refresh, job orchestration) run underneath all of them and are shown after the spine.

1

Raw inputs land

Operator drops source extracts into the input directory. Eleven domains, ordered in `config/etl/etl_config.yaml → domain_order`: item, location, customer, time, sku, sales, forecast, inventory, customer_demand, sourcing, purchase_order.

`data/input/*.csv|*.txt` · `Inventory_Snapshot_YYYY_MM.csv` `dfu_stat_fcst.txt` `sales / masters / PO extracts`

2

Normalize

Each domain's raw file is cleaned and standardized into a staged CSV. The per-domain contract (raw pattern, clean file, target table, keys) lives in one registry.

`data/input/` → `scripts/etl/normalize_dataset_csv.py` `normalize_inventory_csv.py` `normalize_customer_demand_csv.py` → `data/staged/*.csv`

Registry: `common/core/domain_specs.py` (DomainSpec per domain). Never edit staged CSVs by hand — they are regenerated.

3

Load into PostgreSQL

Three modes: `onetime` (first load), `delta` (hash-detect changed files, then COPY → temp table → `INSERT … ON CONFLICT` upsert), `file` (slice replace). The delta upsert pre-filters rows whose foreign keys don't resolve and pre-creates monthly partitions.

`data/staged/` → `scripts/etl/load.py` `load_dataset_postgres.py` →

`dim_item · dim_location · dim_customer · dim_time · dim_sku · dim_sourcing`

→ `fact_sales_monthly` `fact_external_forecast_monthly` `fact_inventory_snapshot ⧉`

`fact_customer_demand_monthly ⧉` `fact_purchase_orders` `fact_open_purchase_orders`

`fact_lead_time_actuals`

Orchestrator: `scripts/etl/run_pipeline.py` (full vs refresh) records batches in `audit_load_batch` for delta detection, then triggers the MV refresh service for exactly the tables written. ⧉ = month-partitioned. Delta upserts require a non-primary unique index per table — guarded by `tests/unit/test_delta_upsert_ddl.py` after today's `sales_ck` incident.

4

SKU features

Statistical descriptors per SKU (demand stats, intermittency, seasonality signals) are computed once and stored on the SKU dimension, where clustering and routing read them.

`fact_sales_monthly` → `scripts/ml/compute_sku_features.py` `common/ml/sku_features/`

→ `dim_sku (feature columns)`

`config/forecasting/sku_features_config.yaml`

5

Clustering

SKUs are grouped into demand-behavior clusters. Scenarios are experiments; promoting one stamps assignments onto the SKU dimension and flags every cluster's tuning profile as stale.

`dim_sku features` → `scripts/ml/run_cluster_pipeline.py` `common/ml/clustering/` →

`cluster_experiment` `dim_sku.ml_cluster` `data/clustering/cluster_labels.csv`

Staleness signal: `promote_scenario()` sets `cluster_tuning_profile_state.stale`. `ml_cluster` is partitioning metadata, never a model feature. A wiped `ml_cluster` silently collapses all per-cluster forecasts — restore via `scripts/ml/restore_cluster_assignments.py`.

6

Per-cluster hyperparameter tuning

Tunes tree hyperparameters per cluster. `--stale-only` re-tunes just the clusters flagged by a clustering promotion, merges into the profile YAML without touching other clusters, stamps the cluster generation, and clears the flags.

`cluster_tuning_profile_state` → `scripts/ml/tune_cluster_hyperparams.py` →

`config/forecasting/cluster_tuning_profiles.yaml`

Consumed by `resolve_cluster_params()` in `common/ml/backtest_framework.py`; `warn_if_profiles_stale()` warns every backtest when profiles are stale or generation-mismatched.

7

Backtest

Rolling-origin backtests per algorithm, trained per cluster with embargo. Every tree instantiation goes through the model registry; every hyperparameter comes from the pipeline YAML.

`fact_sales_monthly + dim_sku` → `scripts/ml/run_backtest.py`

`common/ml/backtest_framework.py` `common/ml/model_registry.py` →

`data/backtest/ results` `data/models/<model>/cluster_N.pkl`

`backtest CSVs` → `scripts/etl/load_backtest_forecasts.py` →

`fact_external_forecast_monthly (per-model rows)` `backtest_lag_archive`

`fact_candidate_forecast`

`config/forecasting/forecast_pipeline_config.yaml`

8

Champion selection

Competes the backtested models per DFU per month (31 strategies) and records the winners. Each experiment is stamped with the clustering generation it was computed under, so downstream stages can refuse cross-generation skew.

`fact_external_forecast_monthly` → `scripts/ml/run_champion_selection.py`

`common/ml/champion/` → `champion_experiment (+cluster_experiment_id)`

`data/champion/experiment_{id}_winners.csv`

`champion rows in fact_external_forecast_monthly`

9

Production forecast — generate, stage, promote

Trains production models on full history, routes each DFU-month to its champion via the winners CSV, applies cold-start and intermittent routing, and writes to staging. Promotion is a deliberate, reviewed API call.

`winners CSV + .pkl models` `dim_sku.ml_cluster` →

`scripts/forecasting/generate_production_forecasts.py` →

`fact_production_forecast_staging`

`staging` → `POST /backtest-management/{model_id}/promote` →

`fact_production_forecast`

Lineage gates: generation preflight in the script (`--allow-cluster-mismatch` to override) and a 409 gate in the promote endpoint when the champion's clustering generation no longer matches the promoted one. Promotion refreshes the production MVs.

10

Inventory planning & operations

Consumes forecasts, inventory snapshots, POs, and lead times to compute policy targets, projections, and exceptions.

`forecasts + inventory + POs` → `scripts/inventory/run_inventory_planning_pipeline.py` → `compute_safety_stock.py · compute_eoq.py · …` → `fact_safety_stock_targets` `fact_eoq_targets`

`compute_inventory_projection.py` → `fact_inventory_projection` ·

`scripts/ops/fix_dq_issues.py` → `fact_dq_check_results`

Inventory math lives in `common/inventory/`; DQ/exception engines in `common/engines/`. Demand signals and replenishment exceptions roll up into the control-tower KPIs.

11

Serve — FastAPI → React

70 mounted routers grouped by domain (`api/routers/{inventory, forecasting, operations, platform, intelligence, core}`, generic `domains.py` catch-all mounted last). The frontend only ever calls `fetchJson` from a query module; Vite proxies each API prefix; types are generated from OpenAPI.

`tables + MVs` → `api/main.py routers` → `frontend/src/api/queries/*.ts` → `frontend/src/tabs/*`

Redis cache (`common/services/cache.py`) with in-memory fallback; seven customer-analytics endpoints opt into the read replica when `READ_REPLICA_URL` is set. The SKU chatbot tab streams over SSE from the intelligence routers using read-only SDK tools.

Cross-cutting service 1 — Materialized-view refresh

One dependency map, one refresh path. Writers never pick MV lists by hand.

common/core/mv_refresh.py

`MV_SOURCES` maps ~30 MVs to their source tables in dependency order. Every writer calls `refresh_for_tables([tables written])` after commit; the dependent closure (including MV-on-MV tiers) is derived, not hand-picked. CLI: `python -m scripts.db.refresh_mvs`. A nightly `refresh_all_mvs` job (02:30, `config/platform/jobs_config.yaml`) bounds any missed path at one day. `mv_intramonth_stockout` is HEAVY (10–30 min) and excluded from derived refreshes by default. Gate rule 7 blocks inline `REFRESH MATERIALIZED VIEW`; `tests/unit/test_mv_refresh.py` fails if a new MV isn't registered.

GROUP	MATERIALIZED VIEWS	REFRESHED WHEN THESE CHANGE
Sales aggregates	agg_sales_monthly · agg_sales_weekly · mv_fill_rate_monthly	fact_sales_monthly, dim_sku
Forecast & accuracy	agg_forecast_monthly · agg_accuracy_by_dim · agg_accuracy_by_dfu · agg_dfu_coverage · agg_dfu_naive_scale · agg_accuracy_lag_archive · agg_dfu_coverage_lag_archive	fact_external_forecast_monthly, backtest_lag_archive, fact_sales_monthly, dim_sku
Customer analytics	mv_customer_activity_monthly · mv_customer_filter_options · mv_ca_segment_trends · mv_ca_demand_at_risk · mv_ca_order_patterns · mv_ca_item_state	fact_customer_demand_monthly, dim_customer, dim_item
Inventory & health	agg_inventory_monthly · mv_inventory_forecast_monthly · mv_network_balance · mv_inventory_health_score · mv_integrated_planning_targets · mv_inventory_projection_summary · mv_intramonth_stockout (heavy)	fact_inventory_snapshot, fact_safety_stock_targets, fact_eoq_targets, fact_inventory_projection
Purchasing	mv_supplier_po_performance · mv_po_lead_time_analysis	fact_purchase_orders
Ops & governance	mv_control_tower_kpis · mv_dq_dashboard · mv_fairness_audit · mv_sensing_overrides_active	exceptions/signals facts, fact_dq_check_results, fact_production_forecast, fact_blended_demand_plan

Cross-cutting service 2 — Job orchestration

One substrate for the whole lifecycle: JobManager on APScheduler, with named pipelines chaining the stages above.

common/services/job_registry.py · job_state.py

~40 registered job types (backtests, champion, tuning, production, inventory, refresh jobs) with per-group FIFO concurrency. Recurring schedules persist in `job_schedule` and are re-registered at API boot. Multi-hour work goes to pg-queue (`common/services/pg_queue.py` + `scripts/ops/pg_queue_worker.py`), not APScheduler. Cross-stage staleness is visible at `GET /dashboard/pipeline-readiness` (cluster wipe, stale tuning, champion lineage, forecast freshness).

NAMED PIPELINE	CHAIN (POST /JOBS/PIPELINES/NAMED/{NAME})
data-refresh	etl_pipeline(refresh) → compute_sku_features → refresh_all_mvs(skip_heavy)
model-refresh	tune_stale_clusters → backtest_lgbm → backtest_catboost → backtest_xgboost → champion_select
forecast-publish	train_production_model(all) → generate_production_forecast — then manual promote
full-refresh	all of the above end-to-end; promotion stays a manual review step

Who reads what — main UI surfaces

TAB (FRONTEND/SRC/TABS)	PRIMARY DATA	ROUTER DOMAIN
ItemAnalysisTab	sales history, staged production forecast (/forecast/production/staging), backtest overlay (/forecast/candidate)	api/routers/forecasting/
AggregateAnalysisTab	accuracy decomposition & error Pareto (agg_accuracy_by_dfu)	api/routers/forecasting/
CustomerAnalyticsTab	mv_customer_activity_monthly fast path (echarts-modular heavy panels)	api/routers/intelligence/customer_analytics/
InvPlanningTab	safety stock / EOQ targets, projections, action feed	api/routers/inventory/ (get_conn pattern)
CommandCenterTab	mv_control_tower_kpis, pipeline readiness	api/routers/core/ + operations/
ClustersTab	cluster_experiment scenarios, promote flow	api/routers/forecasting/
ModelTuningTab	tuning runs & per-cluster profiles	api/routers/forecasting/tuning/
JobsTab	JobManager jobs, named pipelines, schedules	api/routers/core/jobs.py
SkuChatTab	per-SKU conversational assistant (SSE, read-only tools)	api/routers/intelligence/

TAB (FRONTEND/SRC/TABS)	PRIMARY DATA	ROUTER DOMAIN
DataQualityTab	mv_dq_dashboard, DQ auto-fix	api/routers/operations/

Compiled 2026-07-08 from `common/core/domain_specs.py`, `common/core/mv_refresh.py` (authoritative MV map), `config/etl/etl_config.yaml`, `config/forecasting/pipelines.yaml`, `common/services/job_registry.py`, and the ETL/ML/serving modules referenced above. Full architecture: `docs/ARCHITECTURE.md`; operator runbook: `docs/operations-manual/`.